

LAB 3: Component Modelling & Architectural Pattern Selection

Topic: Student Project Portfolio & Showcase App

Name: Rhea Sushanth

SRN: PES1UG24CS371

Github Link: <https://github.com/rheasushanth/SE-Labs-PES1UG24CS371>

Architecture Selection: *"I chose Client-Server Architecture for the Student Project Portfolio & Showcase App."*

1. Architectural Choice

The system is built around one central Application Server that all three user types - Student Team, Panel Judge and Public Voter connect to through a single User Interface Component (the web/app client). The server hosts the Profile Management, Judging, Voting, and Database Components. Together, these components manage showcase profiles, rubric scores, votes, and leaderboard data.

2. Two Scenario-Specific Reasons

Reason 1 – Centralised control is essential for anti-sybil voting: FR-003 and NFR-002 require that exactly one vote per verified identity be accepted and that duplicate/bot votes be detected with 99% accuracy. This is only reliable if one server holds the authoritative, single copy of every voter's identity and vote history. A Microservices split would force this anti-sybil state to be synchronised across services, risking race conditions where a duplicate vote slips through between two nodes; a Client-Server model keeps this check atomic and centralised.

Reason 2 – Simple deployment fits a single-event, single-team build: This is a project built and run by one student team for one expo event, not a large multi-team product. Client-Server lets the whole system be deployed and monitored as one application, which matches the scope and timeline of the project far better than the operational overhead of independently deployed Microservices, while still separating profile, judging and voting logic into distinct components (unlike a purely monolithic Layered build).

3. Security Advantage

Because every client (Student Team, Panel Judge, Public Voter) only ever talks to the server through the User Interface Component and Profile Management Component's IProfileService, no client can write directly to the Database Component. All identity verification (NFR-002's OTP/institutional-SSO check) and duplicate-vote rejection happen server-side inside the Voting Component before any record is persisted, so a malicious or compromised client can, at worst, submit a request - it can never bypass server-side validation or tamper with another user's stored vote or score.

4. Performance Benefit

NFR-001 requires leaderboard updates within 300 ms of a vote or score being submitted. With a centralized server coordinating vote counts and rubric scores, the Database Component can persist the latest results and support rapid leaderboard recalculation without the additional network hops of a distributed microservices architecture."

COMPONENT DIAGRAM

Rhea Sushanth
PES1U0124CS371

Student Project Portfolio & Showcase App Component Diagram (Client - Server Architecture)

